Transl-AI-tor: English-Luxembourgish Translation Using Transfer Learning

Aayam Raj Shakya
shakya.aa@northeastern.edu

Brendan Fullerton
fullerton.b@northeastern.edu

Abhijeet Khandagale
khandagale.a@northeastern.edu

1 Introduction

Despite remarkable advances in neural machine translation (NMT) models, most progress has concentrated on only a few high-resource, widely spoken languages, leaving thousands of others behind. These smaller, underserved languages are commonly described as low-resource languages in natural language processing (NLP) and machine translation (MT). Building an effective translation tool for a low-resource language is technically challenging, yet important for preserving these languages for future generations in the age of AI. Fortunately, several datasets and language models have been developed specifically to support low-resource language work. Achieving effective and accurate translation for such languages, however, remains difficult. This project aims to address that gap by selecting a pre-trained, open-source multilingual translation model—Facebook's No Language Left Behind (NLLB-200) model—and applying transfer learning to refine it for a particular low-resource language, Luxembourgish [2].

In "Overview of Research on Low-Resource Language Machine Translation Based on Artificial Intelligence," Sun et al. discuss several fundamental techniques for MT [15]. Prior to the dominance of Transformer-based models, early translation systems primarily relied on convolutional neural

networks (CNNs) and recurrent neural networks (RNNs). However, RNNs are often poorly suited for complex language tasks because they tend to lose contextual information in longer sequences. To address this problem, long short-term memory (LSTM) models were developed to better handle long-term dependencies. While LSTMs improved performance, they still struggled to preserve context over very long sentences. Furthermore, because LSTMs process data sequentially, they are inherently slower to train. In 2017, researchers at Google transformed the field with the introduction of the Transformer architecture in the paper "Attention Is All You Need" [16]. Unlike traditional RNNs that process sequences linearly, Transformers process tokens in parallel, significantly improving training efficiency. The core of this architecture is the self-attention mechanism, which allows the model to identify and focus on the most relevant tokens regardless of their position in a sentence, aided by positional encodings. The effectiveness of the Transformer architecture led this project to focus on transformer-based models for translation.

Another core MT technique covered by Sun et al. is transfer learning [15]. Transfer learning involves training the model using a higher-resource source language that has a high degree of similarity to the target language in terms of grammar, vocabulary, or syntax. With the high-resource language as the training data (or part of the training data in the case of multilingual models), the model can then be fine-tuned on the low-resource target language. This observation led to a reconsideration of the target language for this project, which was originally going to be Korean. Korean not only uses a different script (Hangul), but also follows a sentence structure that differs from English. English usually follows a Subject-Verb-Object pattern, while Korean follows a Subject-Object-Verb pattern. For example, the English sentence "I eat pizza" translates to "na-neun pija-reul meog-eoyo" in Korean, which literally means "I pizza eat" [1]. Korean is also an agglutinative language, with important semantics often expressed through affixes and non-word morphemes that follow a structure very different from English. Due to these major differences, the project shifted toward another low-resource language that is more similar to English in structure and vocabulary. The target language became Luxembourgish, a low-resource West Germanic language spoken in Luxembourg by around 275,000 people [14]. English is also a West Germanic language, meaning the two languages are distantly related. By the logic of transfer learning, this increased similarity should produce better results.

Based on the findings of Sun et al. and the selection of Luxembourgish as the target language, the project called for a pre-trained multilingual translation model and additional datasets for fine-tuning and evaluation, all of which required availability through Hugging Face. The best fit for this project was Facebook's No Language Left Behind (NLLB-200) model, a multilingual translation system designed specifically for lower-resource languages; Luxembourgish was one of the 200+ languages included in the model's training data, along with English [2]. For fine-tuning, the LuxAlign dataset was selected for its English–Luxembourgish parallel data [10]. For evaluation, the related FLORES+ dataset was used [9], allowing comparison between model output and ground-truth translations. However, when comparing BLEU scores and ChrF scores between the base models and fine-tuned models, no clear differences were shown in the results, bringing the effectiveness of straightforward fine-tuning for low-resource MT into question.

2 Methods

This project was written in Python, using primarily PyTorch and Hugging Face libraries, including `transformers`, `datasets`, `accelerate`, and `evaluate`. Setup involved importing the Facebook NLLB-200-distilled-600M model, LuxAlign dataset, and FLORES+ dataset from Hugging Face, and then creating a `config.py` file to store global hyperparameters [2]. After setup, the first step was to create an evaluation function to measure the model's base performance on Luxembourgish. Translation quality was evaluated using SacreBLEU computed against reference translations from the FLORES+ benchmark. Evaluation began by loading the `devtest` splits from FLORES+ for English and for Luxembourgish. The English texts were set as inputs to the model, which were tokenized using NLLB's multilingual tokenizer [2]. The model was then used to generate predictions by translating each English text into Luxembourgish. Those outputs were compared with the true Luxembourgish texts using the SacreBLEU metric to give the final BLEU score for the base model, which is discussed in the results.

Before fine-tuning, the application deployment and web interface were set up using Gradio, which simplified the process. This required a new `app.py` file that first loaded the models and then

defined interface components along with various input methods, including plain text, image input processed through Tesseract OCR, and audio input transcribed by OpenAI's Whisper small model. The interface and input methods were then tested to ensure they worked correctly with the base model.

Finally, most of the project's development effort was spent implementing the fine-tuning logic. The `finetune` and `eval` flags were used to separate the processes of fine-tuning and evaluation when running the main function. Before incorporating LuxAlign, other datasets were tested for fine-tuning, but most of them led to problems because their formats did not provide parallel English–Luxembourgish data. These issues were resolved once LuxAlign was introduced. Fine-tuning involved splitting the LuxAlign data into training and testing sets, with sizes 10% and 90% respectively; defining a data collator using the NLLB tokenizer; defining a `compute_metrics` function to evaluate training after each epoch; and using Hugging Face's `Seq2SeqTrainer` to train the model. Several issues and pitfalls were encountered while developing the fine-tuning workflow, often related to time and memory limits or `ValueError`s raised by the trainer. Because CUDA provided a more reliable environment, the workflow was ultimately oriented primarily around CUDA-enabled PCs, with CPU execution retained as a fallback. The project initially aimed to support training on Apple's Metal Performance Shaders (MPS) devices as well. In practice, however, repeated memory issues on Mac hardware made this difficult to sustain, even when memory-clearing functions like `torch.mps.empty_cache()` were used. These types of limitations required repeated adjustments to the model arguments, training arguments, and hyperparameters, especially batch size and number of epochs. Training was ultimately conducted for 10 epochs using the Adafactor optimizer, which is a memory-efficient optimizer for large models [5]. To reduce GPU memory overhead, `bf16` mixed precision was used on CUDA-enabled machines [4]. Additional optimizations included bidirectional training by defining train/test datasets for each translation direction (EN→LB and LB→EN) and training both ways, using explicit attention mechanisms such as PyTorch's scaled dot-product attention (SDPA), and implementing layer freezing [12]. FlashAttention 2 was initially defined as the attention mechanism, but it was discarded in favor of SDPA due to incompatibilities among the available CUDA, PyTorch, and Python versions [7]. The use of attention and layer freezing were particularly useful in improving the model's training speed. Bidirectional training was ultimately

discarded, as it had minimal effect on results while simultaneously increasing training time.

3 Results

Both the base model and the fine-tuned model were evaluated against the FLORES+ `devtest` split for Luxembourgish. Translation quality was measured using the SacreBLEU metric, where higher scores indicate closer agreement with the reference translations. A BLEU score between 40% and 60% is generally considered high-quality translation performance, while scores above 60% are particularly strong [8]. Evaluating the base model before fine-tuning produced a BLEU score of 19.6%, well below the benchmark for a good translation. This illustrated the necessity of fine-tuning to achieve a reasonable score. However, upon fine-tuning the model and evaluating again, the score barely changed - through several iterations of fine-tuning and evaluation, the fine-tuned BLEU score did not exceed 20.29%. The size and domain of the fine-tuning data could be a factor in this, as LuxAlign contains text from a Luxembourgish news platform [10]. This writing style may not compare well with FLORES+, which was trained on Wiki sites [9]. However, a more significant factor may be the way in which the results were evaluated, using the BLEU scoring system. While BLEU is effective at measuring interpretability of translated texts in high-resource settings, it tends to penalize morphologically rich languages (such as Luxembourgish) in lower-resource settings due to being an n-gram based metric [6]. Indeed, upon selecting the fine-tuned model that received the 20.29% BLEU score and evaluating it using the character n-gram F-score (ChrF) metric, which measures character-level n-grams, that same model received a score of 51.01%, indicative of a moderately effective translation [11]. Evaluating the base NLLB model using ChrF resulted in a score of 53.62%, even higher than the fine-tuned model. This seemed to indicate that fine-tuning and transfer learning may have limited benefits for low-resource MT when lacking larger and more diverse datasets.

However, the web interface successfully handled all three input modes. Text was extracted from image inputs processed through Tesseract OCR, and audio inputs were transcribed by OpenAI Whisper [13] before being passed to the translation pipeline. In testing, both of these input modes produced

coherent Luxembourgish output. Because no native Luxembourgish speakers were available for evaluation or feedback, the resulting translations were informally double-checked for coherency using Google Translate to convert the Luxembourgish translation back into English. However, Whisper was unable to produce Luxembourgish-to-English translations via audio inputs, and could only translate English audio to Luxembourgish.

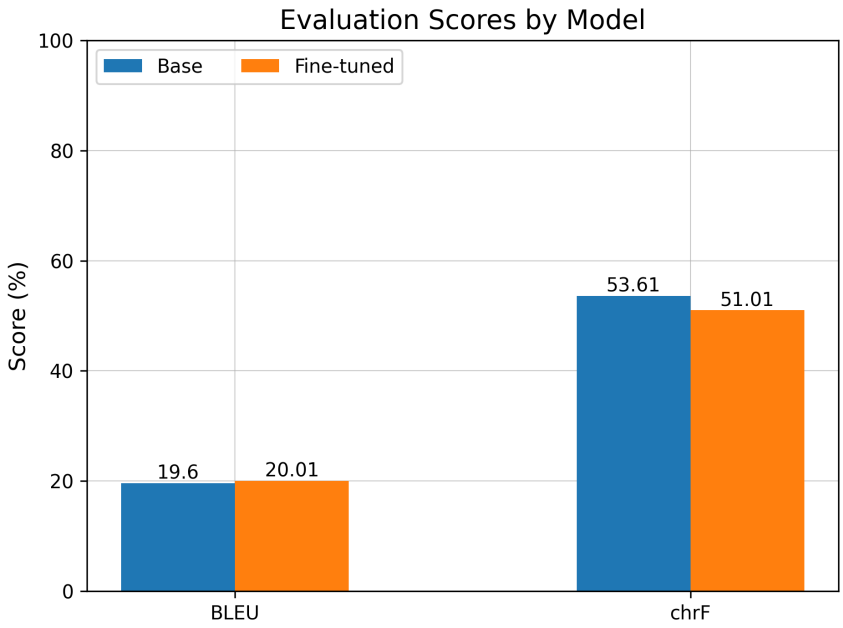
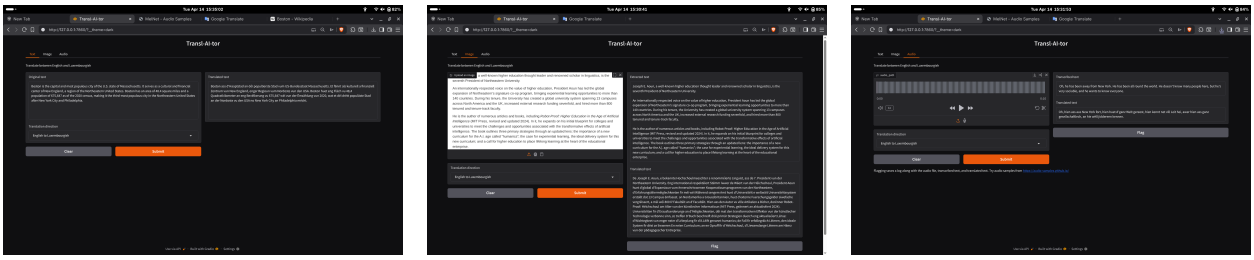
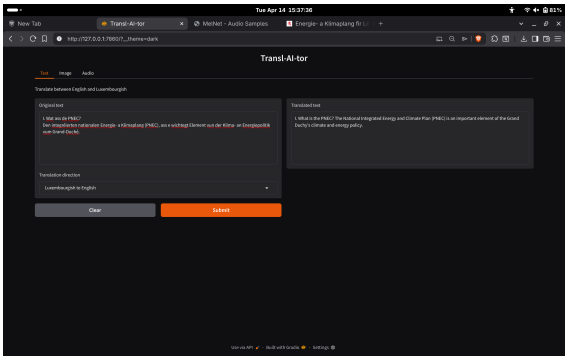


Figure 1: BLEU/ChrF score comparison between the base NLLB model and the fine-tuned model.

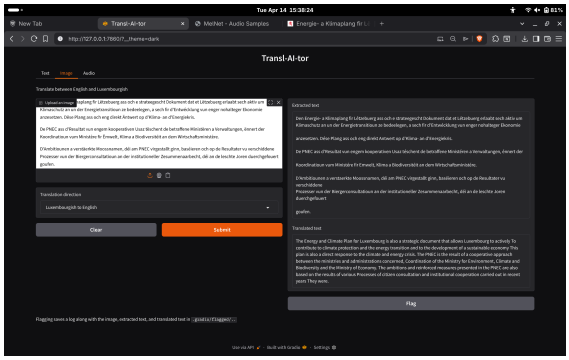


(a) Text input (b) Image input (c) Audio input

Figure 2: English-to-Luxembourgish translations produced by the web interface across text, image, and audio input modes.



(a) Text input



(b) Image input

Figure 3: Luxembourgish-to-English translations produced by the web interface across text and image input modes.

4 Conclusions

The outcomes of this project neatly captured the obstacles and complexities around creating an accurate, user-friendly translator application for a morphologically rich, low-resource language. While the web interface was easy to use and well-integrated with the underlying language models, during evaluation, both the base and fine-tuned models showed moderate performance by ChrF standards and low performance by BLEU standards. These relatively low performances, which were largely unaffected by fine-tuning, were likely due to low data diversity and size. The base NLLB model had already been trained on both English and Luxembourgish, so it is possible that fine-tuning on additional Luxembourgish data made little to no change to the model's underlying structure and knowledge.

The NLLB model and its accompanying FLORES datasets are one of very few resources devoted to low-resource MT, and thus provide a solid base for building an improved translation system. However, since fine-tuning alone failed to improve the accuracy of translations, several additonal steps could be taken in the future to augment the base model. In the case of model training and fine-tuning, data size and diversity does matter, and techniques such as data augmentation and web scraping could be helpful in addressing the difficulties in procuring low-resource langauge data. Data augmentation was intended to be part of the optimization strategy, as it is one of the most useful methods for dealing with low-resource language data, but the time required

to get fine-tuning working prevented its implementation. Web-scraping code was written in the `crawl_site_recursively.py` file, but similarly went unused due to time constraints. If more data were included, along with additonal time and compute, a more powerful model like `nllb-200-3.3B` could achieve better results [3].

However, many of the issues faced in this project surrounding translation accuracy are related less to the quantity of data and more to the data's context and domain, as well as the nature of the Luxembourgish language. Numerical evaluation metrics may be affected by the linguistic context and writing style from which the training data is collected, which may be different compared to that of the evaluation dataset. Additionally, proper care must be taken when deciding on a proper metric to use for evaluating translations of a specific language, particularly in a low-resource domain, demonstrated through the difference between BLEU and ChrF scores. Although Luxembourgish is more similar to English compared to a language like Korean, it is still much more morphologically complex in addition to its low-resource status, as are many other low-resource languages. The project's evaluation was originally based around BLEU scores, until the results prompted a comparison using ChrF, a metric that may have been a better evaluator from the beginning. Most importantly, obtaining feedback from native Luxembourgish speakers is especially important in future projects for understanding the practical quality of the translations, especially when certain evaluation metrics like BLEU and ChrF have inherent trade-offs. Ultimately, the application and model created by this project may serve as a structural foundation for future efforts aimed at translating underserved languages and helping preserve these important cultural resources.

References

[1] Joe Armstrong. Korean sentence structure: The ultimate guide. `https://www.90daykorean.com/korean-sentence-structure/`, March 2026.

[2] Hugging Face. No language left behind. `https://huggingface.co/docs/transformers/en/model_doc/nllb`, 2022.

[3] facebook. nllb-200-3.3b. Hugging Face model card, 2026. NLLB-200 3.3B variant; accessed April 14, 2026.

[4] Hugging Face. Mixed precision training | seq2seqtrainingarguments — Hugging Face transformers documentation. `https://huggingface.co/docs/transformers/en/main_classes/trainer#transformers.Seq2SeqTrainingArguments.bf16`, 2024.

[5] Hugging Face. Optimizer | seq2seqtrainingarguments — Hugging Face transformers documentation. `https://huggingface.co/docs/transformers/en/main_classes/trainer#transformers.Seq2SeqTrainingArguments.optim`, 2024.

[6] Sanjeev Kumar, Preethi Jyothi, and Pushpak Bhattacharyya. Evaluating extremely low-resource machine translation: A comparative study of chrf++ and bleu metrics. *arXiv preprint arXiv:2602.17425*, February 2026.

[7] Lu, Yiren. What is flash attention? `https://modal.com/blog/flash-attention-article`, 2024.

[8] Microsoft. Test your custom translator. `https://learn.microsoft.com/en-us/azure/ai-services/translator/custom-translator/how-to/test-your-model`, 2025.

[9] NLLB Team, Marta R. Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, Anna Sun, Skyler Wang, Guillaume Wenzek, Al Youngblood, Bapi Akula, Loic Barrault, Gabriel Mejia Gonzalez, Prangthip Hansanti, John Hoffman, Semarley Jarrett, Kaushik Ram Sadagopan, Dirk Rowe, Shannon Spruit, Chau Tran, Pierre Andrews, Necip Fazil Ayan, Shruti Bhosale, Sergey Edunov, Angela Fan, Cynthia Gao, Vedanuj Goswami, Francisco Guzmán, Philipp Koehn, Alexandre Mourachko, Christophe Ropers, Safiyyah Saleem, Holger Schwenk, and Jeff Wang. Scaling neural machine translation to 200 languages. *Nature*, 630(8018):841–846, Jun 2024.

[10] Fred Philippy. Luxalign dataset at hugging face. `https://huggingface.co/datasets/fredxlpy/LuxAlign`, 2025.

[11] Maja Popović. chrF: character n-gram F-score for automatic MT evaluation. In Ondřej Bojar, Rajan Chatterjee, Christian Federmann, Barry Haddow, Chris Hokamp, Matthias Huck, Varvara Logacheva, and Pavel Pecina, editors, *Proceedings of the Tenth Workshop on Statistical Machine Translation*, pages 392–395, Lisbon, Portugal, September 2015. Association for Computational Linguistics.

[12] Prabhatzade. Freezing layers and fine-tuning transformer models in pytorch: A simple guide. Medium, 2024.

[13] Vera Shoda. Transcribing audio to text in python using whisper. Medium, 2024.

[14] statistiques.public.lu. Linguistic diversity. `https://statistiques.public.lu/en/publications/recensement/diversite-linguistique.html`, 2026.

[15] Haipeng Sun, Tao Tang, Jiancheng Feng, Zukang Yang, and Bohan Guo. Overview of research on low-resource language machine translation based on artificial intelligence. In *2025 6th International Conference on Computer Engineering and Application (ICCEA)*, pages 01–05. IEEE, 2025.

[16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.